# Lucid : parler à la base de données

Licence professionnelle MMI 3 · Jérémy Chaufourier

# Un ORM, pour quoi faire ?

Manipuler des objets plutôt que d'écrire du SQL à la main

**Sans ORM**

```sql
SELECT * FROM users
WHERE role = 'admin'
ORDER BY created_at DESC
LIMIT 10
```

Puis transformer chaque ligne en objet, à la main, à chaque fois.

**Avec Lucid**

```ts
const admins = await User.query()
  .where('role', 'admin')
  .orderBy('createdAt', 'desc')
  .limit(10)
```

Vous récupérez des `User`, typés, avec leurs méthodes.

> **L'ORM n'est pas là pour vous éviter d'apprendre le SQL.** Il vous évite de le réécrire cent fois. Le jour où une requête devient complexe, vous redescendez au SQL, et il faut savoir le faire.

# Les modèles

Une classe par table, une instance par ligne

### La chaîne complète

**migration** → crée la table `movies`
**database/schema.ts** → `MovieSchema`, généré : les colonnes et leurs types
**app/models/movie.ts** → `Movie`, écrit par vous : ce que la table ne sait pas dire

```
node ace make:model Movie --migration
```

### Les conventions

Modèle `Movie` ↔ table `movies`
Colonne `poster_path` ↔ propriété `posterPath`
Clé primaire `id`, dates `created_at` / `updated_at`

```
// app/models/movie.ts
import env from '#start/env'
import { MovieSchema } from '#database/schema'

export default class Movie extends MovieSchema {
  get posterUrl(): string | null {
    return this.posterPath
      ? `${env.get('TMDB_IMAGE_BASE_URL')}/w500${this.post
      : null
  }

  get releaseYear(): number | null {
    return this.releaseDate?.year ?? null
  }
}
```

**Le modèle, c'est la place de ce qui décrit la donnée** : valeurs calculées, relations, hooks. Pas les règles métier d'un écran, qui vont dans le contrôleur ou un service.

# Lire et écrire

Les opérations qu'on écrit tous les jours

### Lire

```
// Tout
const users = await User.all()

// Un seul, ou une erreur 404
const user = await User.findOrFail(params.id)

// Avec des conditions
const admins = await User.query()
  .where('role', Role.ADMIN)
  .whereNotNull('emailVerifiedAt')
  .orderBy('lastName')
```

### Écrire

```
// Créer
const user = await User.create({
  firstName: 'Ada',
  lastName: 'Lovelace',
  email: 'ada@example.com',
})

// Modifier
user.role = Role.ADMIN
await user.save()

// Supprimer
await user.delete()
```

`node ace repl` permet d'essayer tout ça en direct, sans écrire de contrôleur.

# Les relations

Vos tables sont liées : vos modèles aussi

```ts
// app/models/user.ts
import { hasMany } from '@adonisjs/lucid/orm'
import type { HasMany } from '@adonisjs/lucid/types/relati
import { UsersSchema } from '#database/schema'
import Post from '#models/post'

export default class User extends UsersSchema {
  @hasMany(() => Post)
  declare posts: HasMany<typeof Post>
}
```

```ts
// app/models/post.ts
@belongsTo(() => User)
declare author: BelongsTo<typeof User>
```

**Charger les données liées**

```ts
const user = await User.query()
  .preload('posts')
  .firstOrFail()

console.log(user.posts)
```

```ts
const posts = await Post.query().preload('author')

for (const post of posts) {
  await post.load('author')
  console.log(post.author)
}
// select * from `posts`
// select * from `users` where `id` = ?   ← 50 fois

const posts = await Post.query().preload('author')
// select * from `posts`
// select * from `users` where `id` in (?, ?) ← 1 fois
```

# Remplir la base

Une base vide ne permet de tester ni les écrans, ni les cas limites

**Seeders** : des données de départ

```
// database/seeders/user_seeder.ts
import { BaseSeeder } from '@adonisjs/lucid/seeders'
import User from '#models/user'

export default class extends BaseSeeder {
  async run() {
    await User.createMany([
      { email: 'ada@example.com', password: 'secret' },
      { email: 'alan@example.com', password: 'secret' },
    ])
  }
}
```

```
node ace db:seed
```

**Factories** : des données en masse

Une usine décrit à quoi ressemble un utilisateur crédible : un nom, un email unique, un rôle. On en génère ensuite un, ou mille.

> **Pourquoi c'est indispensable :** une liste testée avec trois lignes paraît toujours parfaite. Avec 500 entrées, on découvre la pagination manquante, les noms trop longs et les temps de chargement.

Seeders et factories font partie du dépôt : n'importe qui clone le projet, lance une commande, et obtient une base utilisable.

# Consommer une API externe

Récupérer des données ailleurs, puis les faire vôtres

```
// database/seeders/movie_seeder.ts
const response = await fetch(
  'https://api.themoviedb.org/3/movie/popular?language=fr-
  {
    headers: {
      Authorization: `Bearer ${env.get('TMDB_TOKEN')}`,
    },
  }
)

const { results } = await response.json()

await Movie.updateOrCreateMany('tmdbId', results.map((m) =
  tmdbId: m.id,
  title: m.title,
  synopsis: m.overview,
  posterPath: m.poster_path,
  releasedAt: DateTime.fromISO(m.release_date),
}))
```

**Trois réflexes**

**Ne stockez jamais la réponse telle quelle.** L'API a sa forme ( `poster_path` , `overview` ), vous avez la vôtre. Traduire à l'entrée, c'est pouvoir changer de fournisseur sans réécrire l'application.

**La clé est un secret.** Elle vit dans `.env` , jamais dans le code, jamais dans le dépôt.

**Une API distante peut tomber.** On importe une fois en base, on travaille en local ensuite. Dépendre du réseau à chaque affichage, c'est dépendre de quelqu'un d'autre pour que son site fonctionne.

# Travaux dirigés

Le ciné-club : remplir la base avec de vrais films

# TD : Les modèles du ciné-club

Trois tables, deux relations, et une clé d'API

### 1. Créer modèles et migrations

```
node ace make:model Movie --migration
node ace make:model Review --migration
```

### 2. Les colonnes

**movies** · tmdb_id (unique), title, synopsis, poster_path, released_at
**reviews** · user_id, movie_id, rating, body

### 3. Les relations

Un film a plusieurs critiques · une critique appartient à un film et à un utilisateur · un utilisateur a plusieurs critiques.

### 4. Importer depuis TMDB

Créez un compte sur **themoviedb.org**, récupérez votre jeton, placez-le dans `.env` :

```
TMDB_TOKEN=votre_jeton
```

```
node ace make:seeder Movie
node ace db:seed
```

> Importez **plusieurs pages** de résultats : viser 200 à 300 films. Avec dix films, tout paraît toujours rapide.

> **Pour aller plus loin :** écrivez votre premier test Japa : une factory qui crée un film, un test qui vérifie qu'il ressort bien de la base.